PROJECT REPORT

Technical Documentation

RAG-Based Customer Support Assistant
with LangGraph & Human-in-the-Loop

Name: Saylee Degloorkar

Intern ID: IN226028702

Date: 22/04/2026

Document Type: Technical Documentation

Table of Contents

Chapter 1: Introduction 3
- 1.1 What is RAG? 3
- 1.2 Why is RAG Needed? 3
- 1.3 Use Case Overview 4

Chapter 2: System Architecture Explanation 5
- 2.1 System Architecture Overview 5
- 2.2 The Five Architecture Layers 5

Chapter 3: Design Decisions 7
- 3.1 Chunk Size: 500 Characters 7
- 3.2 Embedding Strategy 7
- 3.3 Retrieval Approach: K=3, Cosine Similarity 8
- 3.4 Prompt Design 8

Chapter 4: Workflow Explanation 9
- 4.1 Why LangGraph? 9
- 4.2 Node Responsibilities 9
- 4.3 State Transitions 10
- 4.4 LangGraph Workflow Diagram 10

Chapter 5: Conditional Logic 11
- 5.1 Intent Detection 11
- 5.2 Routing Decisions 11
- 5.3 Decision Tree 12

Chapter 6: HITL Implementation 13
- 6.1 Why Human Intervention Matters 13
- 6.2 Benefits and Limitations of HITL 13
- 6.3 HITL Escalation Flow 14
- 6.4 Escalation Triggers 14

Chapter 7: Challenges and Trade-offs 15
- 7.1 Retrieval Accuracy vs. Speed 15
- 7.2 Chunk Size vs. Context Quality 15
- 7.3 Cost vs. Performance 16

Chapter 8: Testing Strategy 17
- 8.1 Testing Approach 17
- 8.2 Sample Test Queries 17
- 8.3 Test Results 18

Chapter 9: Future Enhancements 19
- 9.1 Multi-Document Support 19
- 9.2 Feedback Loop 19
- 9.3 Persistent Memory 19
- 9.4 Cloud Deployment 20
- 9.5 Technical Enhancements 20

Chapter 1: Introduction

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is a way of making AI systems smarter and more reliable. Instead of relying only on what an AI model already knows from training, RAG adds a step where the system first searches a knowledge base for relevant information before it generates an answer. This combination of search and generation is what makes RAG so useful in practice.

There are two main parts to a RAG system:

- Retrieval System – This is like a search engine that looks through stored documents, FAQs, or manuals to find information that matches the user's question.
- Generation System – This is a Large Language Model (LLM) that reads the retrieved information and uses it to write a clear, natural-sounding answer.

The big advantage here is that LLMs are great at producing human-like responses, but they sometimes make things up (this is called 'hallucination'). By giving the model verified information first, RAG helps keep the responses accurate and trustworthy.

In short: without RAG, the AI answers from its training data, which can be outdated or wrong. With RAG, it answers from your specific documents, which are accurate and up to date.

1.2 Why is RAG Needed?

There are a few common problems with using LLMs on their own, and RAG helps solve all of them:

Problem	How RAG Solves It
LLMs can hallucinate facts	Responses are grounded in documents from the knowledge base
Training data goes out of date	The knowledge base can be updated without retraining the model
LLMs lack domain-specific knowledge	Custom PDFs or documents provide the domain expertise needed
No source attribution	Retrieved chunks provide traceable, verifiable sources
Fine-tuning is expensive	RAG is a cheaper and faster alternative for most practical use cases

1.3 Use Case Overview

This project builds a Customer Support Assistant for a fictional company called 'TechFlow Solutions.' The assistant is designed to do a few key things:

- Answer common questions about products, pricing, accounts, and company policies.
- Walk users through troubleshooting steps for technical problems.
- Detect when a query is too complex or sensitive, and escalate it to a human agent.
- Show users which documents were used to form the answer, adding transparency.

Here are a few quick examples of how the system behaves:

- 'How do I reset my password?' → The system retrieves the relevant password reset document and generates a step-by-step answer.
- 'I am very frustrated with your service!' → The system detects this as a complaint and immediately escalates it to a human agent.
- 'What is quantum physics?' → This is outside the knowledge base, so the system flags it as out of scope and escalates it.

Chapter 2: System Architecture Explanation

2.1 System Architecture Overview

The system is built in five layers, each with its own specific job. Together, these layers handle everything from the user typing a question to the final answer being displayed on screen. The architecture is designed to be clean, traceable, and easy to expand in the future.

2.2 The Five Architecture Layers

Layer 1: User Interface

The front end is built using Streamlit, a Python library that makes it easy to create web apps. The chat interface feels natural — users type questions and see responses just like a messaging app. On the side, there is a panel for uploading PDFs and managing the knowledge base. When the system escalates a query to a human, a special escalation panel appears. The interface also shows the source documents that were used to form each answer, so users know where the information is coming from. Chat history is preserved during the session using Streamlit's session state feature.

Layer 2: Workflow Orchestration (LangGraph)

This is the brain of the system. LangGraph manages the entire journey of a query — from the moment a user sends a message to when a response is returned. It does this through a graph of five nodes and two conditional branching points. Every node shares the same state dictionary, which means the workflow is fully traceable and each step can be inspected. The modular design also makes it easy to modify individual steps without breaking the rest of the system.

Layer 3: AI Processing

Two AI models work together in this layer. The first is Groq's Llama 3.3 70B, a large language model that classifies the user's intent and generates the final response. It runs at a temperature of 0.3, which means it stays fairly consistent and factual rather than creative. The second model is HuggingFace's all-MiniLM-L6-v2, which converts text into numerical vectors (called embeddings). It runs locally on the machine, which means no API costs and no network delays. Crucially, both the stored documents and the user's query are converted using this same model, so the comparisons remain meaningful.

Layer 4: Data Storage

ChromaDB is used as the vector database. It stores the embedded text chunks along with their original text and metadata. When a user asks a question, ChromaDB performs a cosine similarity search to find the most relevant chunks. It also saves data to disk, so the knowledge base does not need to be rebuilt every time the app restarts.

Layer 5: HITL (Human-in-the-Loop) System

The escalation engine acts as a safety net. It watches for signals that a query might need human attention — things like a complaint, a low confidence score, or a topic that falls outside the knowledge base. When it detects these signals, it creates a detailed ticket and surfaces an interface where a human agent can type a response. This adds an important layer of quality control to the system.

Chapter 3: Design Decisions

3.1 Chunk Size: 500 Characters

When documents are added to the knowledge base, they are split into smaller pieces called chunks. The chunk size was set to 500 characters, with a 100-character overlap between consecutive chunks (that is, 20% overlap).

Here is why this size was chosen:

- Too small (100–200 characters): The chunks lose context. A question about resetting a password might match a chunk that only contains the word 'passwords,' without any of the actual steps.
- Too large (1,000+ characters): The chunks become noisy and diluted, often containing content about multiple unrelated topics.
- 500 characters (the sweet spot): This is typically enough to capture a full FAQ answer or a complete troubleshooting step.

The 20% overlap ensures that if an important sentence falls on the boundary between two chunks, it still appears fully in at least one of them.

3.2 Embedding Strategy

The embedding model chosen is HuggingFace's all-MiniLM-L6-v2. This model runs entirely on the local machine, which means there are no API costs, no rate limits, and it works even without an internet connection. It produces 384-dimensional vectors, which are compact but still effective at capturing meaning. After the first download (around 80 MB), the model is cached locally for future use.

One important rule: the same embedding model must be used for both storing documents and encoding user queries. If different models were used, the resulting vectors would not be compatible, and similarity searches would give meaningless results.

3.3 Retrieval Approach: K=3, Cosine Similarity

The system retrieves the top 3 most relevant chunks for each query (K=3) using cosine similarity. Cosine similarity is a standard metric for comparing text embeddings because it focuses on the direction of the vectors rather than their size. ChromaDB uses it by default.

As for K=3:

- K=1 is too risky — if the single retrieved chunk is slightly off-topic, the answer will be poor.
- K=3 gives a good balance. If one chunk is incomplete, the others can fill in the gaps.
- K=5 or more tends to introduce noise without meaningfully improving accuracy.

3.4 Prompt Design

The system prompt given to the LLM is built around five key ideas:

1. Identity – The model is introduced as a professional AI Customer Support Assistant.
2. Grounding Rule – The model is told to answer only based on the retrieved context, which prevents hallucinations.

3. Uncertainty Handling – The model is explicitly instructed on what to do when the retrieved context is not sufficient.
4. Tone – The model is asked to be polite, concise, and professional in every response.
5. Formatting – The model is encouraged to use bullet points or numbered steps for clarity.

The temperature is set to 0.3. A value of 0.0 would make the model completely deterministic, while 1.0 or higher would make it overly creative and unpredictable. For a customer support use case, 0.3 strikes the right balance between consistency and natural variation.

Chapter 4: Workflow Explanation

4.1 Why LangGraph?

LangGraph is the orchestration framework used to manage the system's workflow. It supports conditional branching (taking different paths based on what happens at runtime), shared state (all nodes in the graph read from and write to the same dictionary), and a built-in interrupt mechanism for human-in-the-loop scenarios.

A simple sequential chain would process every query the same way, which would not work here. Customer support requires routing: complaints should skip the retrieval step entirely, low-confidence queries should be escalated, and out-of-scope questions need special handling. LangGraph makes all of this explicit and easy to reason about.

4.2 Node Responsibilities

Node	Responsibility	Uses LLM?	Uses DB?
classify_intent	Determines what the user is asking for	Yes	No
retrieve_documents	Finds relevant chunks from the knowledge base	No	Yes
evaluate_confidence	Scores the quality of the retrieved results	No	No
generate_response	Writes the final answer using the retrieved context	Yes	No
escalate_to_human	Creates a ticket and sends the escalation message	No	No

4.3 State Transitions

Normal Flow (FAQ Query)

When a user asks a straightforward question like 'How do I reset my password?', the flow looks like this:

- The classify_intent node identifies the intent as GENERAL_QUERY and sets needs_escalation to false.
- The retrieve_documents node searches ChromaDB and returns the top 3 relevant chunks, along with their confidence scores.
- The evaluate_confidence node checks whether the top score is high enough to proceed.
- The generate_response node uses the retrieved context and the LLM to produce the final step-by-step answer.

Escalation Flow (Complaint)

When a user sends a message like 'I am furious! Your service is terrible!', the flow is different:

- The classify_intent node identifies the intent as COMPLAINT and sets needs_escalation to true.
- The graph skips retrieve_documents and evaluate_confidence entirely.
- The escalate_to_human node creates a ticket and sends an empathetic acknowledgement to the user.

4.4 LangGraph Workflow Diagram

The diagram below shows all five nodes and the two conditional routing paths — the normal flow (retrieval → confidence → response) and the escalation flow (direct to HITL).

LangGraph Workflow — StateGraph with Conditional Routing

```
START → [1. Classify Intent]
                    |
├─ (GENERAL / TECHNICAL / BILLING) → [2. Retrieve Documents] → [3. Evaluate Confidence]
|      ├─ High Confidence → [4. Generate Response] → END
|      └─ Low Confidence → [5. Escalate to Human (HITL)] → END
└─ (COMPLAINT / ESCALATE / OUT_OF_SCOPE) → [5. Escalate to Human (HITL)] → END
```

Chapter 5: Conditional Logic

5.1 Intent Detection

Every query is classified into one of six intent categories by the LLM. Each category triggers a specific action:

Intent	Example Queries	Action Taken
GENERAL_QUERY	'What are your plans?', 'How do I create an account?'	Retrieve → Generate
TECHNICAL_ISSUE	'My login isn't working', 'Video call keeps lagging'	Retrieve → Generate
BILLING	'How do I get a refund?', 'What's on my invoice?'	Retrieve → Generate
COMPLAINT	'I'm very frustrated!', 'Your service is awful'	Escalate immediately
ESCALATE	'Let me talk to a human', 'I need a manager'	Escalate immediately
OUT_OF_SCOPE	'What's the weather?', 'Explain quantum physics'	Escalate immediately

If the LLM returns something unexpected, the system uses keyword-based fuzzy matching to find the closest valid category. As a last resort, it defaults to GENERAL_QUERY.

5.2 Routing Decisions

There are two points in the workflow where the graph decides which path to take:

1. After Intent Classification – If the intent is COMPLAINT, ESCALATE, or OUT_OF_SCOPE, the query goes straight to the human escalation node. This saves the cost of running a retrieval search when it would not help anyway.

2. After Confidence Evaluation – Even if a query passes the first checkpoint, it might still be escalated if the retrieval confidence score is too low (below 0.4) or no relevant documents were found.

5.3 Decision Tree

The routing logic can be summarised as follows:

- User query received → classify intent.
- If COMPLAINT / ESCALATE / OUT_OF_SCOPE → skip retrieval, go to HITL.
- Otherwise → retrieve documents.
- If no documents found → ESCALATE (no context).
- If top confidence score < 0.37 → ESCALATE (low confidence).
- Otherwise → generate response and return it to the user.

Chapter 6: HITL Implementation

6.1 Why Human Intervention Matters

No AI system is perfect, and customer support is one area where getting things wrong can really hurt. The HITL system exists to handle situations where automated responses are not enough:

- Emotional situations: When a customer is frustrated or upset, they need empathy. AI can acknowledge distress, but a human agent is far better at handling these moments with genuine care.
- Edge cases: If a query falls outside what the knowledge base covers, it is better to admit this and hand off to a human than to guess.
- High-stakes decisions: Things like billing disputes, account changes, or anything with legal implications should always have a human in the loop.
- Quality assurance: Humans can catch errors or hallucinations that slip through, keeping the system reliable over time.

6.2 Benefits and Limitations of HITL

Benefits:

Benefit	Why It Matters
Better customer satisfaction	Frustrated users get human attention when they need it most
Risk reduction	Sensitive queries don't get incorrect automated responses
Continuous improvement	Human responses can be used to expand and improve the knowledge base
Builds user trust	Being transparent about AI limitations makes users more comfortable
Legal compliance	Some industries require human oversight for certain decisions

Limitations:

Limitation	Impact
Slower response times	Humans take minutes or hours, while AI responds in seconds
Availability	Human agents are not always online, especially outside business hours
Higher cost	Agent salaries are significantly more expensive than AI processing
Inconsistency	Different agents may give different answers to the same question
Limited scalability	Sudden spikes in queries can overwhelm the support team

6.3 HITL Escalation Flow

When the system decides to escalate, the following steps happen:

1. Trigger detected – The system identifies a complaint, low confidence score, or out-of-scope query.
2. Ticket created – A structured escalation ticket is generated with all the relevant context.
3. Customer notified – The user sees a message explaining that their query is being escalated and a human will follow up.
4. HITL panel shown – A dedicated interface appears in the app where a human agent can type their response.
5. Response integrated – Once the agent replies, the response is added to the chat history and the normal flow resumes.

6.4 Escalation Triggers

The system escalates a query if any one of the following conditions is true:

- The detected intent is COMPLAINT, ESCALATE, or OUT_OF_SCOPE.
- The retrieval confidence score is below 0.4.
- No relevant documents were found in the knowledge base.
- The LLM fails to generate a valid response.

Chapter 7: Challenges and Trade-offs

7.1 Retrieval Accuracy vs. Speed

Approach	Accuracy	Speed	Used?
Exhaustive search	Highest	Slowest	No
Approximate nearest neighbor (HNSW)	High	Fast	Yes
Keyword search (BM25)	Medium	Fastest	No
Hybrid (vector + keyword)	Highest	Moderate	Future enhancement

The system currently uses HNSW-based approximate nearest neighbor search through ChromaDB. This strikes a good balance between accuracy and speed for the current scale of the project.

Chunk Size	Precision	Context Coverage	Embedding Quality
100 characters	High	Very low (fragments)	Poor — too short to be meaningful
300 characters	High	Medium	Good
500 characters (chosen)	Medium-High	Good	Good — captures full answers
1,000 characters	Medium	High	Good but introduces noise
Full page	Low	Very high	Noisy and hard to match accurately

7.3 Cost vs. Performance

LLM	Groq Llama 3.3 70B	GPT-4o	GPT-4o is better for nuanced answers
Embeddings	HuggingFace (local)	OpenAI text-embedding-3	Marginal improvement in accuracy
Vector DB	ChromaDB (local)	Pinecone	Pinecone scales better for production
Hosting	Local machine	Cloud VM (AWS, GCP)	Cloud hosting is needed for production

Chapter 8: Testing Strategy

8.1 Testing Approach

The testing follows a pyramid structure: many unit tests at the bottom, some integration tests in the middle, and a few end-to-end tests at the top. In total, 27 unit tests were written and all of them pass. Here is a breakdown of the test files:

- test_document_processor.py (8 tests): Covers PDF loading, text chunking, and metadata handling.
- test_retriever.py (6 tests): Tests confidence scoring and context formatting.
- test_intent_classifier.py (8 tests): Tests fuzzy matching logic and escalation reason generation.
- test_graph_workflow.py (5 tests): Tests routing logic and state schema validation.

8.2 Sample Test Queries

Query	Expected Intent	Expected Behaviour
'How do I reset my password?'	GENERAL_QUERY	RAG retrieval → confident step-by-step answer
'What subscription plans do you offer?'	GENERAL_QUERY	RAG retrieval → plan details returned
'My video calls keep dropping'	TECHNICAL_ISSUE	RAG retrieval → troubleshooting steps
'I want a refund for last month'	BILLING	RAG retrieval → refund policy explained
'Your service is absolutely terrible!'	COMPLAINT	Immediate HITL escalation
'Let me speak to a manager'	ESCALATE	Immediate HITL escalation
'What is the capital of France?'	OUT_OF_SCOPE	Escalated — outside knowledge base
'How do I export my data?'	GENERAL_QUERY	RAG retrieval → export instructions

Chapter 9: Future Enhancements

9.1 Multi-Document Support

Right now, the system only works with a single PDF as the knowledge base. In the future, it would be useful to support multiple PDFs, with per-document metadata filtering. This would allow users to query specific document sets rather than searching everything at once.

9.2 Feedback Loop

The system currently does not learn from user interactions. Adding a simple thumbs up or thumbs down rating for each response would make it possible to track what is working and what is not. Positive examples could be added to few-shot prompts, and negative examples could highlight gaps in the knowledge base.

9.3 Persistent Memory

At the moment, the conversation history disappears when the user refreshes the page. A future version could use LangGraph's PostgresSaver checkpointer to store conversations across sessions, along with user profiles and conversation summaries for long-term context.

9.4 Cloud Deployment

For production use, the system would need to be containerised with Docker and deployed to a cloud platform such as AWS ECS or Google Cloud Run. This would also require setting up authentication (OAuth), monitoring via LangSmith, and auto-scaling rules based on query volume.

9.5 Technical Enhancements

Several technical improvements could make the system more robust:

- Hybrid Search: Combining vector search with BM25 keyword search would likely improve retrieval accuracy.
- Re-ranking: Adding a cross-encoder model to re-rank retrieved documents before passing them to the LLM.
- Streaming Responses: Streaming the LLM output token by token would make the interface feel more responsive.
- Multi-language Support: Allowing users to ask questions in languages other than English.
- Analytics Dashboard: Tracking query patterns, escalation rates, and response quality over time.
- Automated Knowledge Base Updates: Detecting when documents become outdated and flagging them for review.

End of Report — Version 1.0, April 2026